

Document 4 — Integrating Agents into Mobile & Web Apps

How to consume the deployed LangGraph agents from your own mobile and web applications — securely.

Server: langgraph.sudoxp.com · Repo: github.com/Harikarthik01/LangGraph-Development · Model: claude-sonnet-4-6 · Updated 2026-07-07

We build agents on this LangGraph platform, then consume them from **other applications** (mobile apps, web apps, backends). This guide shows the **correct, secure way** to do that — the #1 rule being: **never put the API key in a client app**.

§0 Contents

- §1 — The golden rule (API-key security)
- §2 — The correct architecture
- §3 — Build a backend proxy (Node & Python)
- §4 — Call it from a web app (React)
- §5 — Call it from a mobile app (React Native, Flutter)
- §6 — Conversation memory (threads)
- §7 — Streaming responses
- §8 — Integration checklist

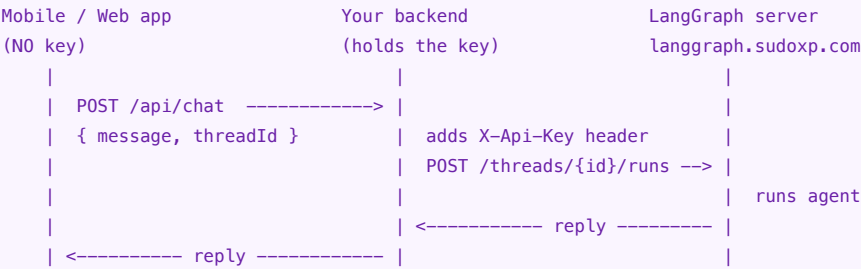
§1 The golden rule: never expose the API key

❌ **NEVER** put the `X-API-Key` in a mobile or web app.

In a **mobile app**, the key can be extracted from the app binary. In a **web app**, it's visible in the browser's Network/Sources tab. Once leaked, anyone can spend your Claude credits.

✅ **The key must live only on a server you control** — your backend — which adds it when forwarding requests to the LangGraph server.

§2 The correct architecture



Layer	Responsibility
Client app (mobile/web)	UI only. Talks to your backend , never to LangGraph directly. Holds NO key.
Your backend (proxy)	Holds the API key (in an env var). Adds <code>X-API-Key</code> , forwards to LangGraph, returns the reply. Add your own user auth here.
LangGraph server	Runs the agent. Only accepts requests carrying the valid key.

Bonus: the backend is also where you add **your app's own user login**, rate limiting, logging, and can hide the LangGraph details from clients entirely.

§3 Build a backend proxy

A thin endpoint that forwards to LangGraph with the key. The key comes from a server env var — never hard-coded.

Node.js (Express)

```
// server.js - run on your own backend
import express from "express";
const app = express();
app.use(express.json());

const LG_URL = "https://langgraph.sudoxp.com";
const LG_KEY = process.env.LANGGRAPH_API_KEY; // set on the server, never in the app

app.post("/api/chat", async (req, res) => {
  // (optional) verify YOUR app's user auth here first
  const { message, threadId } = req.body;

  // create a thread if the client didn't send one
  let tid = threadId;
  if (!tid) {
    const t = await fetch(`${LG_URL}/threads`, {
      method: "POST", headers: { "X-API-Key": LG_KEY, "Content-Type": "application/json" },
      body: "{}",
    }).then(r => r.json());
    tid = t.thread_id;
  }

  const out = await fetch(`${LG_URL}/threads/${tid}/runs/wait`, {
    method: "POST",
    headers: { "X-API-Key": LG_KEY, "Content-Type": "application/json" },
    body: JSON.stringify({ assistant_id: "agent",
      input: { messages: [{ role: "user", content: message }] } }),
  }).then(r => r.json());

  res.json({ threadId: tid, reply: out.messages.at(-1).content });
});

app.listen(3000);
```

Python (FastAPI)

```
import os, requests
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()
LG_URL = "https://langgraph.sudoxp.com"
LG_KEY = os.environ["LANGGRAPH_API_KEY"] # server env var
H = {"X-API-Key": LG_KEY, "Content-Type": "application/json"}

class ChatIn(BaseModel):
    message: str
    threadId: str | None = None

@app.post("/api/chat")
def chat(body: ChatIn):
    tid = body.threadId
    if not tid:
        tid = requests.post(f"{LG_URL}/threads", headers=H, json={}).json()["thread_id"]
    out = requests.post(f"{LG_URL}/threads/{tid}/runs/wait", headers=H, json={
        "assistant_id": "agent",
        "input": {"messages": [{"role": "user", "content": body.message}]},
    }).json()
    return {"threadId": tid, "reply": out["messages"][-1]["content"]}
```

§4 Call it from a web app (React)

The web app calls **your backend** (/api/chat) — not LangGraph. No key in the frontend.

```
async function sendMessage(message, threadId) {
  const res = await fetch("https://your-backend.com/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message, threadId }),
  });
}
```

```
const data = await res.json();
return data; // { threadId, reply } - save threadId for the next message
}
```

CORS: since the app calls *your* backend (same origin or your own CORS rules), you control CORS there — no dependency on the LangGraph server's CORS.

§5 Call it from a mobile app

Same idea — the mobile app calls **your backend**, never LangGraph directly.

React Native

```
const res = await fetch("https://your-backend.com/api/chat", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ message, threadId }),
});
const { threadId: tid, reply } = await res.json();
// store `tid` (e.g. AsyncStorage) to continue the conversation
```

Flutter (Dart)

```
final res = await http.post(
  Uri.parse("https://your-backend.com/api/chat"),
  headers: {"Content-Type": "application/json"},
  body: jsonEncode({"message": message, "threadId": threadId}),
);
final data = jsonDecode(res.body); // { threadId, reply }
```

Native iOS/Android follow the same pattern — a standard HTTPS POST to your backend's `/api/chat`. The mobile app never knows the LangGraph URL or key.

§6 Conversation memory (threads)

To keep context across messages, reuse the `threadId`:

Step	What happens
First message	Client sends no <code>threadId</code> → backend creates one → returns it
Client stores it	Save <code>threadId</code> (localStorage / AsyncStorage / DB)
Next messages	Client sends the same <code>threadId</code> → agent remembers the whole conversation (stored in Postgres)
New conversation	Drop the <code>threadId</code> → backend makes a fresh thread

§7 Streaming responses (typing effect)

For a live "typing" effect, stream tokens. Your backend calls `/runs/stream` and pipes the server-sent events to the client.

```
// backend: proxy the stream
app.post("/api/chat/stream", async (req, res) => {
  const upstream = await fetch(`${LG_URL}/threads/${req.body.threadId}/runs/stream`, {
    method: "POST",
    headers: { "X-API-Key": LG_KEY, "Content-Type": "application/json" },
    body: JSON.stringify({ assistant_id: "agent", stream_mode: "messages",
      input: { messages: [{ role: "user", content: req.body.message }] } }),
  });
  res.setHeader("Content-Type", "text/event-stream");
  upstream.body.pipe(res); // forward the token stream to the client
});
```

The client reads the stream (EventSource / fetch reader) and appends tokens as they arrive — same UX as ChatGPT's live typing.

§8 Integration checklist

- ☐ Client apps call **your backend**, never LangGraph directly
- ☐ The `X-API-Key` lives **only** on your backend (env var), never in app code
- ☐ Your backend adds its own **user authentication** (so only your users can chat)
- ☐ Persist `threadId` per user/conversation for memory

☐ Add rate limiting on your backend (protect Claude usage)

☐ Use HTTPS everywhere (client → backend → LangGraph)

DOC 4 OF 4 Integrating Agents into Apps. See also: Doc 1 (Architecture), Doc 2 (Building Agents), Doc 3 (Setup & Deployment). • Golden rule: client → YOUR backend (holds key) → langgraph.sudoxp.com. Never put the API key in a mobile/web client.